UTEC Posgrado

UTEC Posgrado

UTEC Posgrado

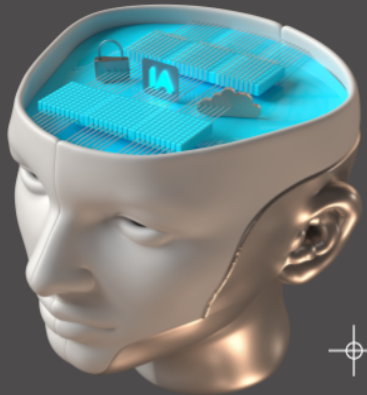
Una escuela para
Early Adopters

MACHINE LEARNING

SUBMUESTREO Y SOBREMUESTREO

Motivación

El problema del **desbalance de clases**



¿Qué es el desbalance de clases?

En muchos problemas reales de clasificación, las clases **no están representadas equitativamente**:

- **Clase mayoritaria**: la que tiene más observaciones

¿Qué es el desbalance de clases?

En muchos problemas reales de clasificación, las clases **no están representadas equitativamente**:

- **Clase mayoritaria**: la que tiene más observaciones
- **Clase minoritaria**: la que tiene pocas observaciones

¿Qué es el desbalance de clases?

En muchos problemas reales de clasificación, las clases **no están representadas equitativamente**:

- **Clase mayoritaria**: la que tiene más observaciones
- **Clase minoritaria**: la que tiene pocas observaciones
- Ratios típicos: 10:1, 100:1, incluso 10000:1

¿Qué es el desbalance de clases?

En muchos problemas reales de clasificación, las clases **no están representadas equitativamente**:

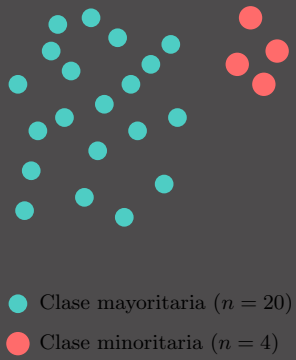
- **Clase mayoritaria**: la que tiene más observaciones
- **Clase minoritaria**: la que tiene pocas observaciones
- Ratios típicos: 10:1, 100:1, incluso 10000:1

▷ Un clasificador ingenuo puede alcanzar alta *accuracy* prediciendo siempre la clase mayoritaria, sin aprender la clase de interés.

Ejemplos de desbalance en la práctica

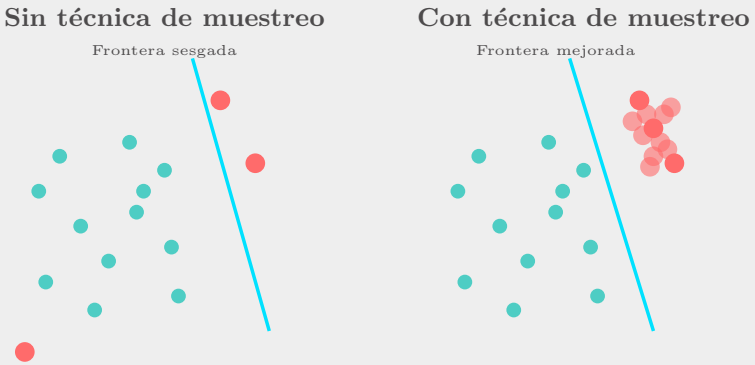
Dominio	Problema	Ratio aprox.
Fraude	Transacciones fraudulentas vs. legítimas	1:1000
Médico	Enfermedad rara vs. pacientes sanos	1:500
Manufactura	Productos defectuosos vs. normales	1:100
Spam	Correos spam vs. legítimos	1:5

Visualización del desbalance



La clase minoritaria (roja) queda "invisible" para el clasificador si no se toman medidas.

Efecto del desbalance en la frontera de decisión



El desbalance sesga la frontera de decisión hacia la clase minoritaria (Hastie et al., 2009, Cap. 4).

Estrategias para manejar el desbalance

1. **A nivel de datos**: modificar la distribución de entrenamiento
   - Submuestreo (undersampling): reducir la clase mayoritaria
   - Sobremuestreo (oversampling): aumentar la clase minoritaria

Estrategias para manejar el desbalance

1. **A nivel de datos**: modificar la distribución de entrenamiento
   - Submuestreo (undersampling): reducir la clase mayoritaria
   - Sobremuestreo (oversampling): aumentar la clase minoritaria
2. **A nivel de algoritmo**: ajustar pesos de clase, costos asimétricos

Estrategias para manejar el desbalance

1. **A nivel de datos**: modificar la distribución de entrenamiento
   - Submuestreo (undersampling): reducir la clase mayoritaria
   - Sobremuestreo (oversampling): aumentar la clase minoritaria
2. **A nivel de algoritmo**: ajustar pesos de clase, costos asimétricos
3. **A nivel de métricas**: usar Precision, Recall, F1, AUC en lugar de accuracy

Estrategias para manejar el desbalance

1. **A nivel de datos**: modificar la distribución de entrenamiento
   - Submuestreo (undersampling): reducir la clase mayoritaria
   - Sobremuestreo (oversampling): aumentar la clase minoritaria
2. **A nivel de algoritmo**: ajustar pesos de clase, costos asimétricos
3. **A nivel de métricas**: usar Precision, Recall, F1, AUC en lugar de accuracy

$\Longrightarrow$ En esta sesión nos enfocamos en las técnicas **a nivel de datos**.

Métricas clave: Matriz de confusión

		Predicho	
		Positivo	Negativo
Real	Positivo	**VP**	**FN**
	Negativo	**FP**	**VN**

- **VP**: Verdaderos Positivos
- **FP**: Falsos Positivos (Error Tipo I)
- **FN**: Falsos Negativos (Error Tipo II)
- **VN**: Verdaderos Negativos

$$\text{Precision} = \frac{VP}{VP + FP} \qquad \text{Recall} = \frac{VP}{VP + FN} \qquad F_1 = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$$

Ejemplo: ¿Por qué accuracy engaña?

Dataset: 1000 muestras, 950 clase negativa, 50 clase positiva.

Modelo que predice siempre "negativo":

	Pred. +	Pred. −
Real +	VP = 0	FN = 50
Real −	FP = 0	VN = 950

Ejemplo: ¿Por qué accuracy engaña?

Dataset: 1000 muestras, 950 clase negativa, 50 clase positiva.

Modelo que predice siempre "negativo":

	Pred. +	Pred. −
Real +	VP = 0	FN = 50
Real −	FP = 0	VN = 950

- Accuracy $= \frac{950}{1000} =$ **95 %** ← Parece excelente!

Ejemplo: ¿Por qué accuracy engaña?

Dataset: 1000 muestras, 950 clase negativa, 50 clase positiva.

Modelo que predice siempre "negativo":

	Pred. +	Pred. −
Real +	VP = 0	FN = 50
Real −	FP = 0	VN = 950

- Accuracy $= \frac{950}{1000} =$ **95 %** ← Parece excelente!
- Recall $= \frac{0}{0+50} =$ **0 %** ← No detecta ningún positivo

Ejemplo: ¿Por qué accuracy engaña?

Dataset: 1000 muestras, 950 clase negativa, 50 clase positiva.

Modelo que predice siempre "negativo":

	Pred. +	Pred. −
Real +	VP = 0	FN = 50
Real −	FP = 0	VN = 950

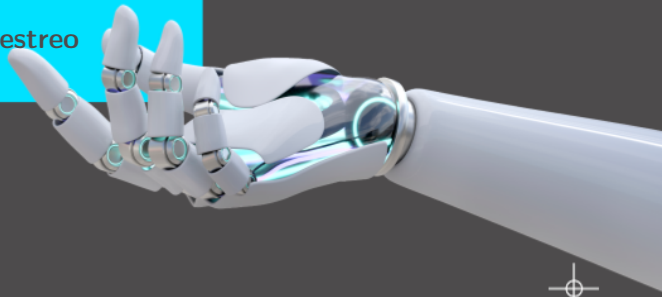
- Accuracy $= \frac{950}{1000} = \mathbf{95\,\%}$ ← Parece excelente!
- Recall $= \frac{0}{0+50} = \mathbf{0\,\%}$ ← No detecta ningún positivo
- $F_1 = \mathbf{0\,\%}$ ← Refleja la realidad

UTEC Posgrado

Una escuela para
Early Adopters

Desarrollo

Técnicas de submuestreo **y sobremuestreo**



Submuestreo (Undersampling): Concepto

El **submuestreo** consiste en **reducir el número de muestras** de la clase mayoritaria para equilibrar la distribución de clases.

- Se eliminan observaciones de la clase con más datos

Submuestreo (Undersampling): Concepto

El **submuestreo** consiste en **reducir el número de muestras** de la clase mayoritaria para equilibrar la distribución de clases.

- Se eliminan observaciones de la clase con más datos
- El objetivo es obtener $|C_{\text{may}}| \approx |C_{\text{min}}|$

Submuestreo (Undersampling): Concepto

El **submuestreo** consiste en **reducir el número de muestras** de la clase mayoritaria para equilibrar la distribución de clases.

- Se eliminan observaciones de la clase con más datos
- El objetivo es obtener $|C_{\text{may}}| \approx |C_{\text{min}}|$
- **Ventaja**: reduce el tiempo de entrenamiento

Submuestreo (Undersampling): Concepto

El **submuestreo** consiste en **reducir el número de muestras** de la clase mayoritaria para equilibrar la distribución de clases.

- Se eliminan observaciones de la clase con más datos
- El objetivo es obtener $|C_{\text{may}}| \approx |C_{\text{min}}|$
- **Ventaja**: reduce el tiempo de entrenamiento
- **Desventaja**: se pierde información potencialmente valiosa

Submuestreo (Undersampling): Concepto

El **submuestreo** consiste en **reducir el número de muestras** de la clase mayoritaria para equilibrar la distribución de clases.

- Se eliminan observaciones de la clase con más datos
- El objetivo es obtener $|C_{\text{may}}| \approx |C_{\text{min}}|$
- **Ventaja**: reduce el tiempo de entrenamiento
- **Desventaja**: se pierde información potencialmente valiosa

Referencia: Hastie et al. (2009), Sección 4.8 y 9.2 — muestreo de casos.

Submuestreo aleatorio: Procedimiento

Algoritmo 1: Submuestreo aleatorio

Entrada: Conjunto de entrenamiento $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$
Salida : Conjunto balanceado $\mathcal{D}'$

Sea $\mathcal{D}_{\text{may}} = \{(x_i, y_i) : y_i = 0\}$ con $|\mathcal{D}_{\text{may}}| = N_0$
Sea $\mathcal{D}_{\text{min}} = \{(x_i, y_i) : y_i = 1\}$ con $|\mathcal{D}_{\text{min}}| = N_1$
Seleccionar aleatoriamente N_1 muestras de $\mathcal{D}_{\text{may}} \Rightarrow \mathcal{D}'_{\text{may}}$
$\mathcal{D}' \leftarrow \mathcal{D}'_{\text{may}} \cup \mathcal{D}_{\text{min}}$
return $\mathcal{D}'$ *con* $|\mathcal{D}'| = 2 \cdot N_1$

Submuestreo: Dataset de ejemplo

Consideremos un dataset de 10 observaciones en $\mathbb{R}^2$:

Obs.	x_1	x_2	Clase y
1	1.0	2.0	0 (may.)
2	1.5	1.8	0 (may.)
3	2.0	2.5	0 (may.)
4	0.5	1.5	0 (may.)
5	1.8	1.0	0 (may.)
6	2.5	2.2	0 (may.)
7	1.2	3.0	0 (may.)
8	4.0	4.0	1 (min.)
9	4.5	3.8	1 (min.)
10	4.2	4.5	1 (min.)

$N_0 = 7$ (mayoritaria), $N_1 = 3$ (minoritaria). Ratio: 7:3 $\approx$ 2,3:1.

Submuestreo aleatorio: Paso a paso

Objetivo: Reducir $\mathcal{D}_{\text{may}}$ de 7 a 3 observaciones.

Paso 1: Identificar clases

- $\mathcal{D}_{\text{may}} = \{1, 2, 3, 4, 5, 6, 7\}$
- $\mathcal{D}_{\text{min}} = \{8, 9, 10\}$
- $N_1 = 3$

Paso 2: Seleccionar $N_1 = 3$ de $\mathcal{D}_{\text{may}}$ al azar.
Supongamos que se seleccionan: $\{2, 5, 6\}$

Paso 3: Formar $\mathcal{D}'$:

Obs.	x_1	x_2	y
2	1.5	1.8	0
5	1.8	1.0	0
6	2.5	2.2	0
8	4.0	4.0	1
9	4.5	3.8	1
10	4.2	4.5	1

$|\mathcal{D}'| = 6$, ratio 1:1 ✓

Submuestreo: Visualización

Las observaciones en gris fueron descartadas al azar. Se pierde información de la clase mayoritaria.

Submuestreo aleatorio: Ventajas y desventajas

Ventajas

- Simple de implementar
- Reduce el tamaño del dataset $\Rightarrow$ entrenamiento más rápido
- No introduce datos artificiales
- Útil cuando $N_0 \gg N_1$

Desventajas

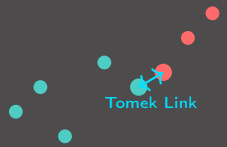
- **Pérdida de información**: muestras potencialmente útiles se eliminan
- Alta varianza: el resultado depende de qué muestras se eligen
- Puede eliminar observaciones cerca de la frontera de decisión

Mitigación: repetir con múltiples submuestras (*ensemble undersampling*).

Submuestreo informado: Tomek Links

Un **Tomek Link** es un par de observaciones (x_i, x_j) de clases distintas tales que no existe ninguna otra observación x_k más cercana a ambas:

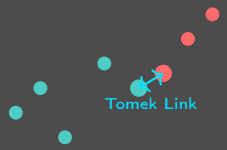
$$d(x_i, x_j) < d(x_i, x_k) \quad \text{y} \quad d(x_i, x_j) < d(x_j, x_k) \quad \forall\, k \neq i, j$$



Submuestreo informado: Tomek Links

Un **Tomek Link** es un par de observaciones (x_i, x_j) de clases distintas tales que no existe ninguna otra observación x_k más cercana a ambas:

$$d(x_i, x_j) < d(x_i, x_k) \quad \text{y} \quad d(x_i, x_j) < d(x_j, x_k) \quad \forall\, k \neq i, j$$

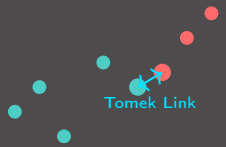


- Se eliminan los pares ambiguos de la frontera

Submuestreo informado: Tomek Links

Un **Tomek Link** es un par de observaciones (x_i, x_j) de clases distintas tales que no existe ninguna otra observación x_k más cercana a ambas:

$$d(x_i, x_j) < d(x_i, x_k) \quad \text{y} \quad d(x_i, x_j) < d(x_j, x_k) \quad \forall\, k \neq i, j$$

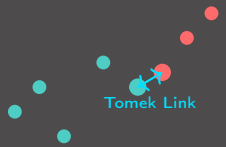


- Se eliminan los pares ambiguos de la frontera
- Limpia la frontera de decisión

Submuestreo informado: Tomek Links

Un **Tomek Link** es un par de observaciones (x_i, x_j) de clases distintas tales que no existe ninguna otra observación x_k más cercana a ambas:

$$d(x_i, x_j) < d(x_i, x_k) \quad \text{y} \quad d(x_i, x_j) < d(x_j, x_k) \quad \forall\, k \neq i, j$$



- Se eliminan los pares ambiguos de la frontera
- Limpia la frontera de decisión
- Más conservador que el aleatorio

Tomek Links: Ejemplo con dataset

Usando nuestro dataset, calculamos las distancias entre clases:

	Obs. 8 (4,0, 4,0)	**Obs. 9** (4,5, 3,8)	**Obs. 10** (4,2, 4,5)
Obs. 1 (1,0, 2,0)	3.61	3.93	4.03
Obs. 2 (1,5, 1,8)	3.12	3.54	3.71
Obs. 3 (2,0, 2,5)	2.50	2.79	2.83
Obs. 6 (2,5, 2,2)	2.33	2.49	2.82
Obs. 7 (1,2, 3,0)	2.95	3.40	3.29

- La distancia más pequeña inter-clase: $d(6, 8) = 2{,}33$
- Si no hay observación x_k más cercana a ambas $\Rightarrow$ Tomek Link
- Se elimina Obs. 6 de la clase mayoritaria

Sobremuestreo (Oversampling): Concepto

El **sobremuestreo** consiste en **aumentar el número de muestras** de la clase minoritaria para equilibrar la distribución.

- Se añaden observaciones a la clase con menos datos

Sobremuestreo (Oversampling): Concepto

El **sobremuestreo** consiste en **aumentar el número de muestras** de la clase minoritaria para equilibrar la distribución.

- Se añaden observaciones a la clase con menos datos
- El objetivo es obtener $|C_{\text{min}}| \approx |C_{\text{may}}|$

Sobremuestreo (Oversampling): Concepto

El **sobremuestreo** consiste en **aumentar el número de muestras** de la clase minoritaria para equilibrar la distribución.

- Se añaden observaciones a la clase con menos datos
- El objetivo es obtener $|C_{\text{min}}| \approx |C_{\text{may}}|$
- **Ventaja**: no se pierde información de la clase mayoritaria

Sobremuestreo (Oversampling): Concepto

El **sobremuestreo** consiste en **aumentar el número de muestras** de la clase minoritaria para equilibrar la distribución.

- Se añaden observaciones a la clase con menos datos
- El objetivo es obtener $|C_{\text{min}}| \approx |C_{\text{may}}|$
- **Ventaja**: no se pierde información de la clase mayoritaria
- **Desventaja**: puede generar *overfitting* si solo se duplica

Sobremuestreo (Oversampling): Concepto

El **sobremuestreo** consiste en **aumentar el número de muestras** de la clase minoritaria para equilibrar la distribución.

- Se añaden observaciones a la clase con menos datos
- El objetivo es obtener $|C_{\text{min}}| \approx |C_{\text{may}}|$
- **Ventaja**: no se pierde información de la clase mayoritaria
- **Desventaja**: puede generar *overfitting* si solo se duplica

▷ El enfoque más directo es la **duplicación aleatoria** de observaciones minoritarias.

Sobremuestreo aleatorio: Procedimiento

Algoritmo 2: Sobremuestreo aleatorio

Entrada: $\mathcal{D}_{\text{min}}$ con N_1 muestras, $\mathcal{D}_{\text{may}}$ con N_0 muestras
Salida : Conjunto balanceado $\mathcal{D}'$

Calcular $M = N_0 - N_1$ (muestras adicionales necesarias)
for $j = 1$ **to** M **do**
  Seleccionar aleatoriamente $x_k \in \mathcal{D}_{\text{min}}$ (con reemplazo)
  Añadir una copia de (x_k, y_k) a $\mathcal{D}_{\text{min}}$
end
$\mathcal{D}' \leftarrow \mathcal{D}_{\text{may}} \cup \mathcal{D}_{\text{min}}$
return $\mathcal{D}'$ *con* $|\mathcal{D}'| = 2 \cdot N_0$

Sobremuestreo aleatorio: Paso a paso

Objetivo: Aumentar $\mathcal{D}_{\text{min}}$ de 3 a 7 observaciones ($M = 7 - 3 = 4$).

Paso 1: Identificar

- $\mathcal{D}_{\text{min}} = \{8, 9, 10\}$
- $N_0 = 7$, necesitamos $M = 4$ copias

Paso 2: Muestrear con reemplazo:

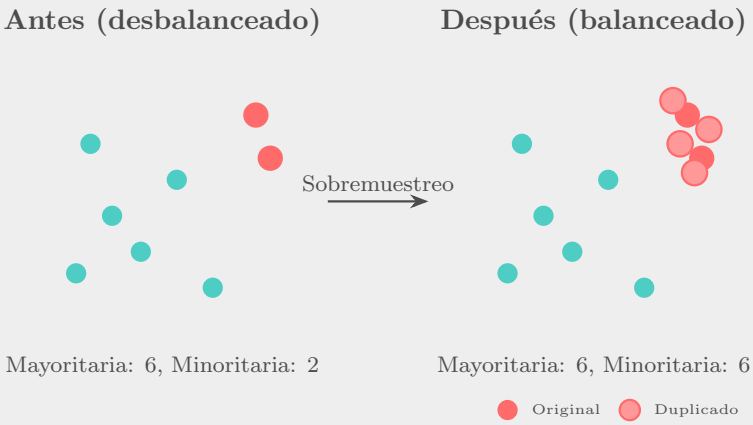
- $j = 1$: copiar Obs. 8
- $j = 2$: copiar Obs. 10
- $j = 3$: copiar Obs. 9
- $j = 4$: copiar Obs. 8

Resultado:

Obs.	x_1	x_2	y	Nota
8	4.0	4.0	1	original
9	4.5	3.8	1	original
10	4.2	4.5	1	original
8'	4.0	4.0	1	copia
10'	4.2	4.5	1	copia
9'	4.5	3.8	1	copia
8''	4.0	4.0	1	copia

$|\mathcal{D}'_{\text{min}}| = 7$, ratio 1:1 ✓

Sobremuestreo: Visualización



Los puntos con borde rojo son copias exactas. **Problema:** el clasificador memoriza

Problema del sobremuestreo aleatorio

Al duplicar observaciones idénticas:

- Las regiones de decisión se "endurecen" alrededor de los puntos existentes

Problema del sobremuestreo aleatorio

Al duplicar observaciones idénticas:

- Las regiones de decisión se "endurecen" alrededor de los puntos existentes
- No se agrega **información nueva**

Problema del sobremuestreo aleatorio

Al duplicar observaciones idénticas:

- Las regiones de decisión se "endurecen" alrededor de los puntos existentes
- No se agrega **información nueva**
- El modelo memoriza los pocos ejemplos minoritarios

Problema del sobremuestreo aleatorio

Al duplicar observaciones idénticas:

- Las regiones de decisión se "endurecen" alrededor de los puntos existentes
- No se agrega **información nueva**
- El modelo memoriza los pocos ejemplos minoritarios
- Alto riesgo de **overfitting**

Problema del sobremuestreo aleatorio

Al duplicar observaciones idénticas:

- Las regiones de decisión se "endurecen" alrededor de los puntos existentes
- No se agrega **información nueva**
- El modelo memoriza los pocos ejemplos minoritarios
- Alto riesgo de **overfitting**

Mitigación: usar *ensemble oversampling* (múltiples muestras con reemplazo), ajustar pesos de clase, o combinar con submuestreo.

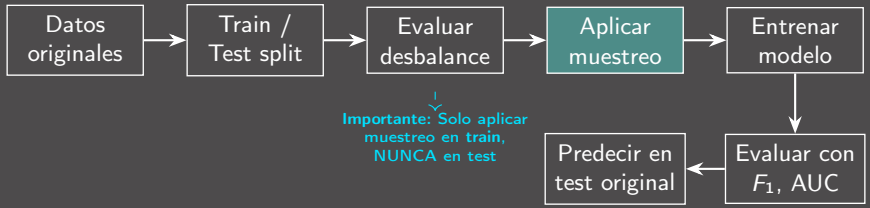
Comparación de técnicas de muestreo

Aspecto	Sub. aleatorio	Sobre. aleatorio
Información	Pierde datos	No pierde
Overfitting	Bajo	Alto
Datos nuevos	No	No (copias)
Complejidad	$O(1)$	$O(1)$
Tamaño final	Pequeño	Grande
Ruido	No agrega	Puede amplificar

¿Cuándo usar cada técnica?

Escenario	Recomendación
Dataset muy grande	Submuestreo aleatorio
Dataset pequeño	Sobremuestreo aleatorio
Frontera de decisión ruidosa	Submuestreo con Tomek Links
Muchas características	Submuestreo aleatorio
Desbalance moderado	Sobremuestreo aleatorio
Necesidad de interpretabilidad	Submuestreo (datos reales)

Pipeline completo de manejo de desbalance



Regla de oro: El muestreo se aplica **solo al conjunto de entrenamiento**. El conjunto de test debe mantener la distribución original para una evaluación realista.

Ejemplo integrado: Comparación de resultados

Con nuestro dataset de ejemplo ($N_0 = 7$, $N_1 = 3$), entrenando un clasificador:

Técnica	Precision	Recall	F_1
Sin muestreo	1.00	0.33	0.50
Submuestreo	0.60	1.00	0.75
Sobremuestreo	0.75	1.00	0.86

- Sin muestreo: alta precision pero bajo recall (no detecta minoritaria)
- Sobremuestreo logra mejor balance entre precision y recall

Recalibración de probabilidades: Ejemplo

Dataset original: $\pi_0 = 7/10 = 0{,}70$, $\pi_1 = 3/10 = 0{,}30$
Después de submuestreo: $\tilde{\pi}_0 = 3/6 = 0{,}50$, $\tilde{\pi}_1 = 3/6 = 0{,}50$

Si el modelo entrenado con datos balanceados predice $\hat{P}(Y = 1|X = x^*) = 0{,}6$:

$$\hat{P}_{\text{corr}}(Y{=}1|x^*) = \frac{0{,}6 \cdot \frac{0{,}30}{0{,}50}}{0{,}6 \cdot \frac{0{,}30}{0{,}50} + 0{,}4 \cdot \frac{0{,}70}{0{,}50}}$$

$$= \frac{0{,}6 \cdot 0{,}60}{0{,}6 \cdot 0{,}60 + 0{,}4 \cdot 1{,}40} = \frac{0{,}36}{0{,}36 + 0{,}56} = \frac{0{,}36}{0{,}92} \approx \mathbf{0.391}$$

La probabilidad corregida (0.391) es menor que la no corregida (0.6), reflejando la verdadera prevalencia de la clase minoritaria.

Caso 1: Detección de fraude bancario

Problema: Un banco tiene 2 millones de transacciones diarias, de las cuales solo 200 son fraudulentas (ratio 10000:1). El modelo actual predice todo como "legítimo" y obtiene 99.99 % de accuracy.

Opción A: Sobremuestreo aleatorio

- Duplicar las 200 transacciones fraudulentas hasta igualar los 2 millones
- Ratio de duplicación: $\times 10000$

Opción B: Submuestreo aleatorio

- Reducir las transacciones legítimas a 200
- Dataset final: 400 observaciones

Caso 1: Respuesta — Submuestreo aleatorio

Respuesta correcta: Opción B (submuestreo aleatorio).

¿Por qué B es mejor?

- Con 2 millones de muestras legítimas, hay **información redundante**; submuestrear no pierde patrones clave
- Se puede usar *ensemble undersampling*: crear múltiples submuestras y promediar
- Entrenamiento mucho más rápido

¿Por qué A es mala?

- Duplicar 200 muestras $\times 10000$ genera **overfitting extremo**: el modelo memoriza las pocas transacciones fraudulentas
- No agrega información nueva
- Dataset resultante enorme (4 millones) sin beneficio real

Caso 1: ¿Y si no muestreamos?

No muestrear no es viable en este caso.

- Con ratio 10000:1, cualquier clasificador estándar **ignora la clase minoritaria** por completo
- Incluso algoritmos con `class_weight` tienen dificultades: solo 200 positivos entre 2 millones genera gradientes dominados por la clase mayoritaria
- El accuracy "perfecto" de 99.99 % prediciendo todo como legítimo demuestra que el modelo **no aprende** la clase de interés
- **Conclusión:** con desbalance extremo (>1000:1), alguna forma de muestreo es prácticamente necesaria

Caso 2: Diagnóstico de enfermedad rara

Problema: Un hospital tiene registros de 500 pacientes: 475 sanos y 25 con una enfermedad rara (ratio 19:1). Cada registro tiene 50 variables clínicas. Se necesita un modelo con alto recall para no dejar pasar ningún caso positivo.

Opción A: Submuestreo aleatorio

- Reducir sanos de 475 a 25
- Dataset final: 50 observaciones

Opción B: Sobremuestreo aleatorio

- Duplicar los 25 enfermos hasta tener 475
- Dataset final: 950 observaciones

Caso 2: Respuesta — Sobremuestreo aleatorio

Respuesta correcta: Opción B (sobremuestreo aleatorio).

¿Por qué B es mejor?

- El dataset ya es **pequeño** (500); submuestrear dejaría solo 50 muestras, insuficiente para 50 variables
- Conserva toda la información de los 475 pacientes sanos
- Aunque duplica, el modelo al menos aprende la frontera con todos los datos disponibles

¿Por qué A es mala?

- 50 observaciones con 50 variables $\Rightarrow$ **más variables que datos** ($p \geq N$)
- El modelo no puede generalizar
- Se descartan 450 registros valiosos de pacientes sanos que definen la normalidad

Caso 2: ¿Y si no muestreamos?

No muestrear podría funcionar parcialmente, pero con limitaciones.

- Con 500 muestras y ratio 19:1, un modelo con `class_weight` podría aprender algo, pero los **25 casos positivos son muy pocos** para definir una frontera robusta
- Algoritmos como árboles de decisión con pocas hojas podrían ignorar la clase minoritaria al buscar pureza global
- Sin muestreo, el recall sería bajo: el requisito clínico de "no dejar pasar ningún caso" exige reforzar la clase minoritaria
- **Conclusión:** en datasets pequeños con pocos positivos y alto costo de error, el sobremuestreo sigue siendo preferible a no hacer nada

Caso 3: Control de calidad en manufactura

Problema: Una fábrica inspecciona 100,000 piezas al mes. El 1 % son defectuosas (1000 defectuosas vs. 99,000 normales). Los sensores capturan 5 variables por pieza. La frontera entre "normal" y "defectuosa" es difusa: muchos productos están en el límite.

Opción A: Submuestreo aleatorio

- Reducir normales de 99,000 a 1,000
- Dataset final: 2,000 observaciones

Opción B: Submuestreo con Tomek Links

- Eliminar solo los pares ambiguos de la frontera
- Dataset final: ∼95,000 observaciones

Caso 3: Respuesta — Submuestreo con Tomek Links

Respuesta correcta: Opción B (Tomek Links).

¿Por qué B es mejor?

- La frontera es **difusa**: hay muchos puntos ambiguos que confunden al clasificador
- Tomek Links elimina **selectivamente** los pares conflictivos, limpiando la frontera
- Conserva la gran mayoría de datos informativos

¿Por qué A es mala?

- Al reducir de 99,000 a 1,000 **aleatoriamente**, se pueden eliminar puntos clave de la frontera
- Con frontera difusa, se necesitan muchos datos cerca del límite para aprender la separación
- Alta varianza en los resultados

Caso 3: ¿Y si no muestreamos?

No muestrear es una alternativa razonable en este caso.

- Con 100,000 muestras y ratio 99:1, hay **1,000 defectuosas**: suficientes para que muchos clasificadores aprendan
- Modelos como Random Forest o Gradient Boosting con `class_weight` manejan bien este nivel de desbalance
- Sin embargo, la frontera **difusa** sigue siendo un problema: sin limpiar los puntos ambiguos, el modelo puede cometer errores sistemáticos en la zona límite
- **Conclusión:** no muestrear es viable, pero Tomek Links aporta una ventaja específica al limpiar la frontera — si la frontera fuera nítida, no muestrear sería suficiente

Caso 4: Filtro de spam en correo electrónico

Problema: Un proveedor de correo tiene 10,000 correos etiquetados: 8,000 legítimos y 2,000 spam (ratio 4:1). El modelo se actualiza cada hora con nuevos datos. Se prioriza velocidad de entrenamiento sin perder mucha precisión.

Opción A: Sobremuestreo aleatorio

- Duplicar spam de 2,000 a 8,000
- Dataset final: 16,000 observaciones

Opción B: Submuestreo aleatorio

- Reducir legítimos de 8,000 a 2,000
- Dataset final: 4,000 observaciones

Caso 4: Respuesta — Submuestreo aleatorio

Respuesta correcta: Opción B (submuestreo aleatorio).

¿Por qué B es mejor?

- El modelo se reentrena **cada hora**: la velocidad es crítica
- 4,000 muestras entrenan mucho más rápido que 16,000
- El desbalance es moderado (4:1); 2,000 muestras por clase son suficientes
- Se puede variar la submuestra en cada reentrenamiento

¿Por qué A es mala?

- Cuadruplica el dataset $\Rightarrow$ entrenamiento $4\times$ más lento
- Incompatible con el requisito de reentrenamiento frecuente
- Duplicar spam introduce riesgo de overfitting a patrones específicos de spam antiguo

Caso 4: ¿Y si no muestreamos?

No muestrear es una buena alternativa aquí.

- El desbalance es **moderado** (4:1) con 2,000 muestras de spam: suficientes para la mayoría de clasificadores
- Con `class_weight` o ajuste de umbral, el modelo puede compensar el desbalance sin modificar datos
- **Sin embargo**, el requisito de velocidad sigue vigente: entrenar con 10,000 muestras es más lento que con 4,000
- **Conclusión:** no muestrear funciona bien en términos de calidad del modelo, pero el submuestreo se prefiere aquí **por eficiencia computacional**, no por el desbalance en sí

Caso 5: Predicción de abandono de clientes (churn)

Problema: Una empresa de telecomunicaciones tiene 50,000 clientes: 45,000 activos y 5,000 que abandonaron (ratio 9:1). El costo de no detectar un abandono es 10× mayor que una falsa alarma. Se dispone de tiempo y recursos computacionales amplios.

Opción A: Submuestreo aleatorio

- Reducir activos de 45,000 a 5,000
- Dataset final: 10,000 observaciones

Opción B: Sobremuestreo aleatorio

- Duplicar abandonos de 5,000 a 45,000
- Dataset final: 90,000 observaciones

Caso 5: Respuesta — Sobremuestreo aleatorio

Respuesta correcta: Opción B (sobremuestreo aleatorio).

¿Por qué B es mejor?

- Hay **tiempo y recursos** suficientes; no importa el dataset grande
- Conserva los 45,000 perfiles de clientes activos: información valiosa para distinguir patrones
- El costo asimétrico exige maximizar recall; más datos ayudan a capturar todos los patrones de abandono

¿Por qué A es mala?

- Descartar 40,000 clientes activos elimina **perfiles diversos** de no-abandono
- El modelo pierde capacidad de distinguir falsos positivos
- Con costo asimétrico alto, se necesita la mejor frontera posible, y eso requiere más datos, no menos

Caso 5: ¿Y si no muestreamos?

No muestrear es viable, pero subóptimo dado el contexto.

- Con ratio 9:1 y 5,000 positivos, clasificadores como XGBoost o SVM con `class_weight` podrían funcionar bien sin muestreo
- Sin embargo, el **costo asimétrico** ($10\times$) demanda maximizar recall: el sobremuestreo fuerza al modelo a prestar más atención a los patrones de abandono
- Sin muestreo, el modelo tiende a optimizar accuracy global, lo cual puede sacrificar recall de la clase minoritaria
- **Conclusión:** no muestrear no es mala opción si se combinan pesos de clase y umbral; pero con costo asimétrico alto y recursos disponibles, el sobremuestreo ofrece una capa adicional de seguridad

Caso 6: Clasificación de correos prioritarios

Problema: Una empresa clasifica correos entrantes como "urgente" (5 %) o "normal" (95 %). El dataset tiene 1,000 correos (50 urgentes, 950 normales). Sin embargo, el clasificador basado en regresión logística ya permite ajustar el **umbral de decisión** y asignar **pesos por clase**. El desbalance es moderado (19:1).

Opción A: Submuestreo aleatorio

- Reducir normales de 950 a 50
- Dataset final: 100 observaciones

Opción B: Sobremuestreo aleatorio

- Duplicar urgentes de 50 a 950
- Dataset final: 1,900 observaciones

Caso 6: Respuesta — Ninguna técnica de muestreo

Respuesta correcta: Ni A ni B. Usar **pesos de clase** y **ajuste de umbral**.

¿Por qué no muestrear?

- La regresión logística acepta el parámetro `class_weight` que penaliza más los errores en la clase minoritaria
- Ajustar el umbral de 0,5 a un valor menor (ej. 0,2) aumenta el recall sin modificar los datos
- Se conserva **toda la información** y la **distribución real**
- No se introduce sesgo ni overfitting

¿Por qué A y B son malas aquí?

- **Submuestreo**: descarta 900 correos normales; con solo 100 datos el modelo no generaliza
- **Sobremuestreo**: duplicar 50 correos urgentes $\times 19$ causa overfitting severo a esos pocos ejemplos
- Ambas técnicas **distorsionan** las probabilidades calibradas que la regresión logística produce naturalmente

Discusión: ¿El muestreo funciona igual en todo clasificador?

No. El efecto del muestreo depende fuertemente del tipo de clasificador:

Clasificador	Efecto positivo del muestreo	Efecto negativo del muestreo
Regresión logística	Mejora recall cuando no se dispone de `class_weight`	Distorsiona las probabilidades calibradas; requiere recalibración
SVM	Submuestreo reduce el costo $O(N^2–N^3)$; mejora la frontera	Sobremuestreo duplica vectores de soporte idénticos sin beneficio
Árboles / Random Forest	Los árboles se benefician del muestreo por bootstrap; mejora splits	Sobremuestreo puede generar splits artificiales en datos duplicados
k-NN	Submuestreo reduce el ruido de la clase mayoritaria en la vecindad	Sobremuestreo crea vecindarios densos artificiales; sesga las votaciones

Discusión: Clasificadores afectados positiva y negativamente

Se benefician del muestreo:

- **SVM**: el submuestreo reduce drásticamente el tiempo de entrenamiento ($O(N^3)$) y elimina puntos redundantes lejos de la frontera
- **k-NN**: al submuestrear, la clase mayoritaria deja de "inundar" las vecindades, permitiendo que la minoritaria tenga voto justo
- **Redes neuronales**: el sobremuestreo equilibra los batches de entrenamiento, evitando gradientes dominados por una clase

Pueden verse perjudicados:

- **Regresión logística**: ya estima $P(Y|X)$ directamente; muestrear distorsiona las probabilidades y requiere recalibración posterior
- **Naive Bayes**: altera las frecuencias a priori $P(Y = k)$; las estimaciones de verosimilitud se sesgan con datos duplicados
- **Gradient Boosting**: ya maneja desbalance internamente con la función de pérdida; el muestreo puede interferir con el proceso de boosting secuencial

Discusión: Lecciones de las 6 casuísticas

1. **No existe una regla universal**: la decisión de muestrear depende del **tamaño del dataset**, el **ratio de desbalance**, el **clasificador** y los **requisitos del negocio**

Discusión: Lecciones de las 6 casuísticas

1. **No existe una regla universal**: la decisión de muestrear depende del **tamaño del dataset**, el **ratio de desbalance**, el **clasificador** y los **requisitos del negocio**
2. **No muestrear es válido** cuando el clasificador tiene mecanismos internos (pesos, umbral) y el desbalance es moderado (casos 3, 4, 6)

Discusión: Lecciones de las 6 casuísticas

1. **No existe una regla universal**: la decisión de muestrear depende del **tamaño del dataset**, el **ratio de desbalance**, el **clasificador** y los **requisitos del negocio**
2. **No muestrear es válido** cuando el clasificador tiene mecanismos internos (pesos, umbral) y el desbalance es moderado (casos 3, 4, 6)
3. **El muestreo es necesario** cuando el desbalance es extremo (>100:1) o el dataset es muy pequeño (casos 1, 2)

Discusión: Lecciones de las 6 casuísticas

1. **No existe una regla universal**: la decisión de muestrear depende del **tamaño del dataset**, el **ratio de desbalance**, el **clasificador** y los **requisitos del negocio**
2. **No muestrear es válido** cuando el clasificador tiene mecanismos internos (pesos, umbral) y el desbalance es moderado (casos 3, 4, 6)
3. **El muestreo es necesario** cuando el desbalance es extremo (>100:1) o el dataset es muy pequeño (casos 1, 2)
4. **Siempre evaluar**: probar con y sin muestreo usando validación cruzada estratificada y métricas como F_1 o AUC, nunca accuracy

Discusión: Lecciones de las 6 casuísticas

1. **No existe una regla universal**: la decisión de muestrear depende del **tamaño del dataset**, el **ratio de desbalance**, el **clasificador** y los **requisitos del negocio**
2. **No muestrear es válido** cuando el clasificador tiene mecanismos internos (pesos, umbral) y el desbalance es moderado (casos 3, 4, 6)
3. **El muestreo es necesario** cuando el desbalance es extremo (>100:1) o el dataset es muy pequeño (casos 1, 2)
4. **Siempre evaluar**: probar con y sin muestreo usando validación cruzada estratificada y métricas como F_1 o AUC, nunca accuracy
5. **El clasificador importa**: antes de muestrear, verificar si el algoritmo elegido ya ofrece soluciones internas al desbalance

Conclusiones

Resumen y reflexiones **Próximos pasos**

Resumen: Submuestreo y sobremuestreo

1. **Desbalance de clases**: problema común que sesga los clasificadores

Resumen: Submuestreo y sobremuestreo

1. **Desbalance de clases**: problema común que sesga los clasificadores
2. **Submuestreo aleatorio**: reduce la clase mayoritaria; simple pero pierde información

Resumen: Submuestreo y sobremuestreo

1. **Desbalance de clases**: problema común que sesga los clasificadores
2. **Submuestreo aleatorio**: reduce la clase mayoritaria; simple pero pierde información
3. **Tomek Links**: submuestreo informado que limpia la frontera

Resumen: Submuestreo y sobremuestreo

1. **Desbalance de clases**: problema común que sesga los clasificadores
2. **Submuestreo aleatorio**: reduce la clase mayoritaria; simple pero pierde información
3. **Tomek Links**: submuestreo informado que limpia la frontera
4. **Sobremuestreo aleatorio**: duplica la minoritaria; riesgo de overfitting

Resumen: Submuestreo y sobremuestreo

1. **Desbalance de clases**: problema común que sesga los clasificadores
2. **Submuestreo aleatorio**: reduce la clase mayoritaria; simple pero pierde información
3. **Tomek Links**: submuestreo informado que limpia la frontera
4. **Sobremuestreo aleatorio**: duplica la minoritaria; riesgo de overfitting
5. **Recalibración**: necesaria cuando se modifican las proporciones de clase

Buenas prácticas

1. **Siempre evaluar el desbalance** antes de entrenar

Buenas prácticas

1. **Siempre evaluar el desbalance** antes de entrenar
2. **Muestrear solo en train**: nunca modificar el conjunto de test

Buenas prácticas

1. **Siempre evaluar el desbalance** antes de entrenar
2. **Muestrear solo en train**: nunca modificar el conjunto de test
3. **Usar métricas adecuadas**: F_1, AUC-ROC, no accuracy

Buenas prácticas

1. **Siempre evaluar el desbalance** antes de entrenar
2. **Muestrear solo en train**: nunca modificar el conjunto de test
3. **Usar métricas adecuadas**: F_1, AUC-ROC, no accuracy
4. **Validación cruzada estratificada**: mantener proporciones en cada fold

Buenas prácticas

1. **Siempre evaluar el desbalance** antes de entrenar
2. **Muestrear solo en train**: nunca modificar el conjunto de test
3. **Usar métricas adecuadas**: F_1, AUC-ROC, no accuracy
4. **Validación cruzada estratificada**: mantener proporciones en cada fold
5. **Combinar técnicas**: submuestreo + sobremuestreo, o Tomek Links + sobremuestreo

Buenas prácticas

1. **Siempre evaluar el desbalance** antes de entrenar
2. **Muestrear solo en train**: nunca modificar el conjunto de test
3. **Usar métricas adecuadas**: F_1, AUC-ROC, no accuracy
4. **Validación cruzada estratificada**: mantener proporciones en cada fold
5. **Combinar técnicas**: submuestreo + sobremuestreo, o Tomek Links + sobremuestreo
6. **Recalibrar probabilidades** si se usan para toma de decisiones

Referencias

- Hastie, Tibshirani & Friedman (2009). *The Elements of Statistical Learning*, 2da ed. Springer. Cap. 4, 7, 9, 10.
- He, H. & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Trans. on Knowledge and Data Engineering*, 21(9), 1263–1284.

UTEC Posgrado

UTEC Posgrado